

Lab 5 Practice: Random Memory Access

Objective

This exercise introduces you to the architecture and usage of random-access memory (RAM). You will design and verify a **2-read, 1-write (2R1W) RAM** that supports random memory access. The goal is to understand how data can be efficiently read and written at arbitrary locations, a key concept for high-performance systems and algorithms.

Action Items

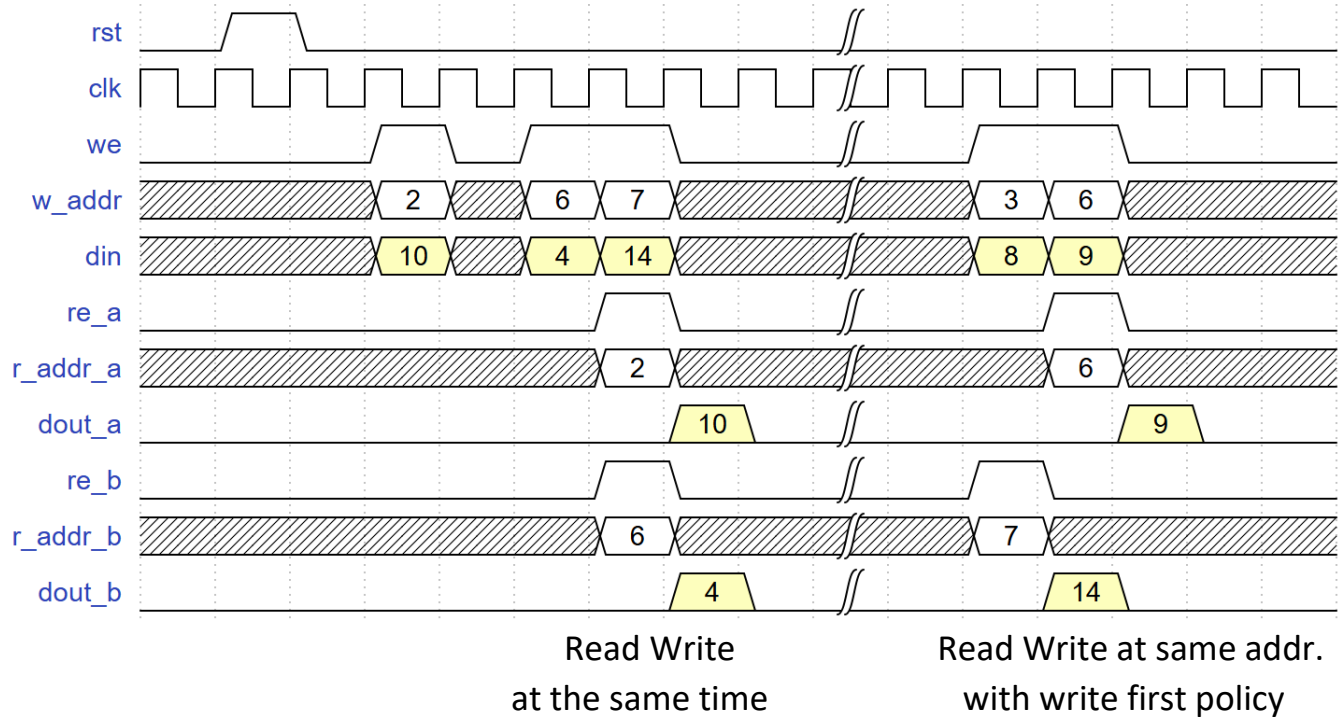
Implement a 2R1W memory array consisting of eight 4-bit entries, with write-first policy and two simultaneous read ports.

IO Specifications:

Signal	I/O	Width	Description
clk	Input	1	Positive-edge triggered clock signal
rst	Input	1	Synchronous active-high reset: all memory entries are cleared to zero when rst ==1'b1
we	Input	1	Write enable: write the data-in (din) into the memory when we ==1'b1
w_addr	Input	3	Write address: the location in memory for write operation
din	Input	4	Value to be written into the w_addr position of the memory
re_a, re_b	Input	1	Read enables for ports A/B: when asserted (re_a ==1'b1 or re_b ==1'b1), the corresponding port outputs the data from the memory.
r_addr_a, r_addr_b	Input	3	Read addresses for ports A and B: the locations in memory to read for ports A and B
dout_a, dout_b	Output	4	Data outputs from r_addr_a/r_addr_b positions in the memory for ports A/B

Functionality:

- The memory of eight 4-bit entries can be modeled as
 `reg [3:0] memory [0:7];`
in Verilog.
- Upon reset, all memory contents are cleared to zero. To reset the memory content, you can utilize a for-loop statement to initialize all entries in your Verilog code.
- Read operation:
 - o When asserted **re_a**==1'b1 or **re_b**==1'b1, the corresponding port outputs the data from the memory.
 - o Signals **dout_a**/**dout_b** output zero when **re_a**/**re_b** == 1'b0.
 - o Read operations are synchronized with the positive clock edges.
- Write operation:
 - o When **we** == 1'b1, the value of **din** is written to the address **w_addr** on the rising clock edge.
 - o **Write-First Policy**: if a write and a read target the same address in the same clock cycle, the read port should immediately return the **newly written data (din)**.
 - o Write operations are also synchronized with the positive clock edges.
- Create your own testbench to write the data into the memory at arbitrary addresses. Additionally, you can overwrite the existing values.
- Read out the values stored in the memory to verify correctness.
- Refer to the following timing diagram as an example.
Note that the slashed regions in the diagram represent **don't-care values**, not **unknown values**, meaning the logic levels at these points are irrelevant to the behavior of the design.



Use the following template as the basis for your design:

```

module lab5_practice (
    input wire clk,
    input wire rst,
    // write port
    input wire we,
    input wire [2:0] w_addr,
    input wire [3:0] din,
    // read port A
    input wire re_a,
    input wire [2:0] r_addr_a,
    output reg [3:0] dout_a,
    // read port B
    input wire re_b,
    input wire [2:0] r_addr_b,
    output reg [3:0] dout_b
);
    // Add your design here
    // Note that you are free to adjust the IO's data type
endmodule

```

Attention

- If you create several modules, merge them into a single Verilog file.
- This practice prepares you for the next lab. Make sure you fully understand your design.
- Feel free to ask questions about the specification on the EECLASS forum.